

Evaluation of Reward Functions for Robots with Natural Language Instructions

1st Anika Datta
Independent Researcher

2nd Anandaswarup Vadapalli
Research Spark Hub Inc.
<https://orcid.org/0009-0009-7685-089X>

Abstract—Reinforcement learning turns a specification of what is good into behaviour, but misspecified rewards can produce unpredictable failures. This challenge is especially acute for natural-language tasks, where instructions such as “go behind the sofa” do not directly define a computable reward. We compare three reward sources for language-conditioned navigation in a shared 2-D grid world: a hand-written geometric rule, a large language model (LLM) rubric-based judge, and a small reward network distilled from the LLM.

We evaluate four controlled settings: zero-shot rule-vs-judge comparison (E1), instruction-conditioned iterative judge refinement (E2), refinement against an imperfect REINFORCE-trained policy (E3), and reward-model distillation (E4). E1–E3 each evaluate 64 situations, while E4 evaluates 48 held-out situations using a network trained on 258 LLM judgments. All LLM judgments are real Claude Haiku 4.5 calls.

The zero-shot LLM judge agrees with the hand-written policy’s best action 78% of the time (95% CI [67%, 87%]); one to two rounds of feedback increase agreement to 100%. Against the imperfect policy, agreement reaches 86% (CI [75%, 92%]). The distilled reward network closely reproduces LLM rubric scores (MAE 0.058, CI [0.048, 0.068]) with roughly 1800× lower median latency, but matches the LLM’s best-action choice only 48% of the time (CI [34%, 62%]). This gap between rubric-level and action-level fidelity highlights the need to evaluate distilled reward models at the level of downstream decisions, not only score prediction.

Keywords—Reinforcement learning, reward functions, large language models, reward modeling, reward distillation, language-conditioned navigation

I. INTRODUCTION

Reinforcement learning (RL) turns a specification of “what is good” into behaviour, by having an agent maximise the reward it receives while interacting with an environment [1]. This makes the reward function the single most consequential design choice in an RL system: whatever objective is written down, the agent will optimise, including any loopholes it contains. A poorly specified reward is readily exploited, producing policies that accumulate high reward while failing the intended task - the phenomenon of reward hacking [2].

Reward specification becomes markedly harder when the task is given in natural language rather than as a fixed goal state. A hand-written reward can be made precise, cheap, and reproducible, but it typically bypasses the language entirely, computing success from privileged simulator state, and must be re-derived by hand for every new kind of instruction. Two alternatives have emerged. The first delegates reward computation to a large language model, either by having it write reward *code* [3], [4] or by having it act as a *judge*

that scores behaviour directly, in the spirit of learning from preference feedback [5] and of using one model to grade another’s output [6]. The second distills whatever signal the first produces into a small, fast model queryable without further API calls. Each option trades off precision, cost, and coverage differently, and to our knowledge no prior study holds one task, environment, and instruction set fixed while directly comparing all three.

This paper reports such a comparison. We build one shared gridworld navigation task and run four controlled experiments, E1 through E4, that isolate exactly one variable at a time: whether the reward source is a rule or a judge (E1), whether the judge additionally receives language and is allowed to iteratively refine its answer (E2), whether the policy it is asked to track is near-optimal or genuinely imperfect (E3), and whether the judge itself can be replaced by a distilled network without losing fidelity (E4). Unlike a purely descriptive design document, every number we report is produced by code that actually executes, including real calls to an LLM judge for every situation evaluated in E1–E4. The remainder of the paper reviews prior work on reward specification and LLM-based judging (Section II), states what is novel and what this paper contributes (Sections III and IV), describes the shared environment and the exact protocol for each experiment (Section V), reports and analyses the results of E1–E4 (Section VI), and concludes with implications and limitations (Section VII).

II. PREVIOUS WORK

Hand-designed rewards and reward hacking. The traditional approach to reward design is to write a reward function by hand, typically combining a sparse task-completion bonus with a dense shaping term; potential-based reward shaping [7] showed that a broad class of such terms can be added without changing the set of optimal policies, which is why distance-to-goal shaping remains a standard navigation baseline. Such rewards are precise and cheap, but computed from privileged simulator state rather than the task description, and every new instruction type requires new code. A large body of work has also documented that RL agents readily exploit gaps between a reward function and the behaviour it was meant to encode [2], which motivates evaluating a reward not only by the return it induces but by whether the action preferences it induces match an independently-computed notion of correctness, the comparison at the centre of our E1–E3 protocol.

Reward from human and AI feedback. Christiano et al. [5] showed that a reward model trained from pairwise human preferences can substitute for a hand-written reward in deep RL, at the cost of needing many preference queries. Using an LLM as an automatic *judge* in place of a human is now common outside RL as well: Zheng et al. [6] showed that a sufficiently capable LLM’s pairwise judgments correlate well with human preference on open-ended text, which is the same evaluate-and-trust pattern we apply to a judge scoring a robot’s rubric.

LLMs as reward designers. Most closely related to this paper is a recent line of work in which an LLM produces a reward function rather than a single reward value. Eureka [3] uses an LLM to write and iteratively refine reward *code* for continuous control tasks, evaluating candidate reward functions by the RL policies they induce. Text2Reward [4] similarly generates dense, Python-executable reward functions from a natural-language task description and a symbolic environment representation. Both approaches keep an LLM in the design loop but move it *out* of the per-episode reward computation once a reward function has been selected. Our LLM judge instead stays in the loop at inference time - it scores each situation directly, as a rubric-based judge rather than a code generator - which lets us study its raw agreement with a reference policy (E1) and its steerability via feedback (E2-E3) directly, before asking whether that judge can be distilled away (E4).

Instruction-conditioned navigation. BabyAI [8] and its successor MiniGrid [9] popularised compact, partially-observable gridworlds for language-conditioned policies, typically paired with a frozen sentence encoder such as Sentence-BERT [10] and trained with PPO [11] via, e.g., Stable-Baselines3 [12]. Our gridworld follows this tradition in spirit - a compact grid, named objects, a discrete action space - but keeps the environment and policies deliberately lightweight (Section V) so that every reward source, including a real LLM judge, can be evaluated directly rather than only through the proxy of a fully trained policy’s return.

III. NOVELTY

To our knowledge, this is the first study to compare a hand-written rule, a real (non-mocked) LLM judge, and a network distilled from that judge, on *one* task, environment, and instruction set, rather than studying each reward source in a separate paper and task family. Three aspects of the design are specifically new relative to the work reviewed above. First, where Eureka [3] and Text2Reward [4] use an LLM to write a reward function once and then evaluate the RL policies it produces, we keep the LLM in the loop as a per-situation judge and measure its raw agreement with a reference policy directly, without the confound of a full training run. Second, we introduce an *iterative refinement protocol* (E2, E3) in which the judge is told how far its action distribution is from a reference policy and is re-queried; measuring how many rounds this takes, and whether the judge converges to whichever reference it is given even when that reference

is a genuinely suboptimal 72%-success neural network (E3), is a diagnostic not previously reported for LLM-as-judge reward systems. Third, our distillation study (E4) separates two notions of fidelity - rubric-score agreement and action-level (argmax) agreement - that a single aggregate error metric would conflate, showing they can diverge sharply (0.058 mean absolute rubric error versus 48% action agreement on the same held-out situations, $n = 48$).

IV. CONTRIBUTION

Concretely, this paper contributes:

- A shared, fully specified 2-D gridworld environment (walls, target and distractor objects, a four-action space, and a four-field rubric) used identically across all four experiments (Section V).
- Four controlled experiments, E1-E4, each varying exactly one factor - reward source, presence of language, quality of the reference policy, and presence of the LLM at inference time - so that their results are directly comparable (Sections V-VI).
- An iterative LLM-judge refinement protocol with a measurable convergence criterion (Jensen-Shannon divergence to a reference action distribution), applied to both a near-optimal rule-based policy and a deliberately imperfect learned policy.
- A quantitative account of the trade-off between distilling an LLM judge into a fast reward network and preserving its judgments, reported separately at the rubric level and the action level.
- An open, reproducible codebase in which every reported number comes from an executed run, including real (non-mocked) LLM API calls for every judged situation.

V. EXPERIMENTAL SETUP

A. Shared environment

All four experiments share one 8×8 grid (Fig. 1) with a border wall, two interior wall bands (each with a single one-cell gap), a target object (“sofa”), and two distractor objects (“chair”, “cube”). Of the 64 cells, 26 are free and, by construction (verified with breadth-first search), fully connected to the target. The action space is the four cardinal moves {up, right, down, left}; an action that would cross a wall or the grid boundary is a no-op and is logged as an *obstacle hit*. This is a deliberately lightweight stand-in for a full MiniGrid-style turn-based agent [9], keeping every reward source, including a real LLM judge, cheap enough to query for every situation in every experiment (Section VII).

Every experiment operates on *situations*: tuples of (current cell, steps taken so far, obstacles hit so far). A situation is produced by rolling a reference policy forward from a start cell for roughly half of its shortest-path distance to the target, so that situations are realistic and mid-episode rather than only trivial starts or already-solved cases. Sixteen of the 26 free cells are at least 3 moves from the target; rather than sampling a handful of them once, every eligible cell is rolled out four independent times with distinct random seeds, giving

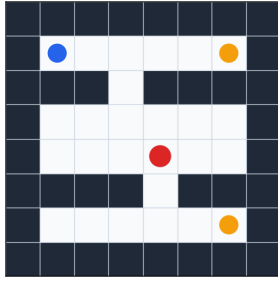


Fig. 1. The shared 8×8 gridworld. Dark cells are walls; the red disc is the target (“sofa”); orange discs are distractors (“chair”, “cube”); the blue disc marks the agent’s start cell.

64 situations per experiment for E1-E3 (three replicates, 48 situations, for E4’s held-out set) - a statistically reasonable sample from this map, discussed further as a limitation in Section VII.

For a given situation, two quantities are computed. The first is an *action-probability distribution* over the four moves. The second is a four-field *rubric*: whether the target is reached, a closeness score (0-1, based on shortest-path distance to the target), an avoided-damage score (0-1, penalised by obstacle hits), and a path-efficiency score (0-1, optimal path length over steps actually taken). Every reward source in this paper - rule-based, LLM judge, neural-network policy, and distilled reward network - is asked to produce, or is scored against, these same two quantities, which is what makes their outputs directly comparable across E1-E4.

B. The rule-based policy

The rule-based policy computes both quantities directly from grid geometry, in the potential-based-shaping spirit of a hand-designed distance reward [7]: action probabilities are a softmax over the reduction in breadth-first-search distance to the target each move would produce, and the rubric fields follow from the same distances. It requires no learning or API calls and serves as the near-optimal reference policy in E1 and E2.

C. The neural-network policy

For E3 we train a small 2-layer multilayer perceptron (16 hidden units, tanh activation, softmax output over the four moves) with REINFORCE and a dense, distance-shaped reward, for 1,500 episodes. This policy reaches a 72% success rate at reaching the target within 40 steps - a deliberately imperfect reference, in contrast to the near-optimal rule-based policy, that lets us test whether the LLM judge can be steered toward a policy that is not, itself, correct.

D. The LLM judge

For a given situation, the LLM judge (Claude Haiku 4.5, via the Anthropic API) is given: (a) an ASCII rendering of the map with the agent’s cell marked, (b) two rendered images - the agent’s current view and the goal configuration - and, in E2-E4, (c) an instruction such as “Go to the sofa. Avoid

bumping into walls and take the shortest path you can.” It also receives the raw step and obstacle-hit counters. A JSON tool-use schema constrains the model to return the same four rubric fields used by the rule-based policy plus an action-probability distribution over the four moves and a short free-text justification. Every call in E1-E4 is a genuine network call, not a cached or mocked response.

E. E1: zero-shot judge vs. rule-based policy

For all 64 situations, the judge is given no instruction and is called once per situation, with exactly the information available to the rule-based policy, isolating its raw, language-free agreement with a geometric reference.

F. E2: language and iterative refinement, rule-based reference

For a fresh set of 64 situations, the judge additionally receives an instruction and is iteratively re-queried: after each answer we compute the Jensen-Shannon (JS) divergence between its action distribution and the rule-based policy’s and, if it exceeds $\tau = 0.05$, feed that divergence and the reference distribution back as text and ask the model to reconsider, for up to 4 rounds - testing whether feedback is a cheap, effective alignment mechanism, and how many rounds it takes.

G. E3: language and iterative refinement, neural-network reference

The protocol is identical to E2 on another fresh set of 64 situations, but the policy the judge is steered toward is the 72%-success neural network (Section V-C) rather than the near-optimal rule - testing whether refinement still converges against an imperfect target, and whether doing so costs agreement with the independently-correct rule-based answer.

H. E4: distilling a reward network

We pool every (situation, LLM rubric) pair from E1-E3 - 258 pairs, including every intermediate refinement round - and fit a small regression network (two hidden layers, 32/16 units, tanh) to predict the four rubric fields from six state features (normalised position, offset to target, obstacle-hit count, step count). Action probabilities are then derived from predicted closeness the same way the rule-based policy derives them from true distance: a softmax over the predicted closeness of each next state. We evaluate on 48 fresh held-out situations (three replicates per eligible cell), each also given one fresh real LLM call, so the network’s output can be compared directly against a judgment it never trained on.

VI. RESULTS AND ANALYSIS

A. E1: rule-based policy vs. zero-shot LLM judge

The LLM judge agrees with the rule-based policy’s preferred action 78% of the time (50/64 situations, 95% CI [67%, 87%]), with mean cosine similarity 0.878 and mean JS divergence 0.095 (CI [0.062, 0.128]) between the two action distributions (Table I); rubric agreement is closer still (mean absolute error 0.053, CI [0.044, 0.062]). Disagreements cluster at cells adjacent to a wall gap, where the judge sometimes

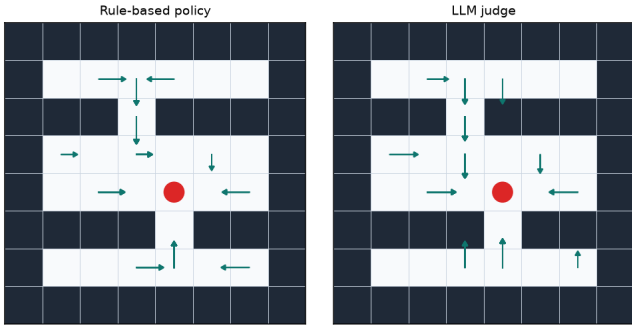


Fig. 2. E1: preferred action (arrow) at each sampled cell, rule-based policy (left) vs. zero-shot LLM judge (right), no instruction.

misjudges which side of a wall segment is passable despite the attached images - for example, at one cell the rule-based policy places 0% probability on the “down” move because it is blocked, while the judge assigns it 0.55. Fig. 2 overlays the two policies’ preferred action at every cell visited across the 64 situations; the two fields agree almost everywhere except at these wall-adjacent cells. This suggests that a modern LLM’s failure mode on this task is a specific, correctable perceptual error (reading the map), not a conceptual misunderstanding of what “closer to the target” means.

B. E2: language and iterative refinement toward the rule-based policy

All 64 situations converge within the 4-round budget (convergence rate 100%, CI [94%, 100%]), in a mean of 1.34 rounds (CI [1.23, 1.46]). Final-round agreement with the rule-based policy rises sharply relative to E1: mean cosine similarity 0.998, mean JS divergence 0.017 (CI [0.014, 0.020]), and 100% argmax agreement (Table I). The single largest case is a cell where a zero-shot JS divergence of 0.43 falls to 0.01 after just one round of feedback. The effect is not merely the judge restating what the images already showed: the same wall-adjacent cells that were misjudged in E1 are corrected within one refinement round once the divergence and reference distribution are given as explicit text feedback, which indicates that language plus a concrete feedback signal is doing real corrective work.

C. E3: language and iterative refinement toward an imperfect neural-network policy

Convergence is measurably harder against the imperfect reference: mean iterations rises from 1.34 (E2) to 1.69 (CI [1.51, 1.87]) - confidence intervals that do not overlap. All 64 situations still eventually converge within the round budget. Final agreement with the neural-network policy is nonetheless high (cosine similarity 0.993, argmax agreement 86%, CI [75%, 92%]) - but the same converged judgments now agree with the independently-computed rule-based policy at only 0.768 cosine similarity, down from 0.998 in E2. In other words, the judge is not converging toward some policy-independent notion of correctness; it is converging toward

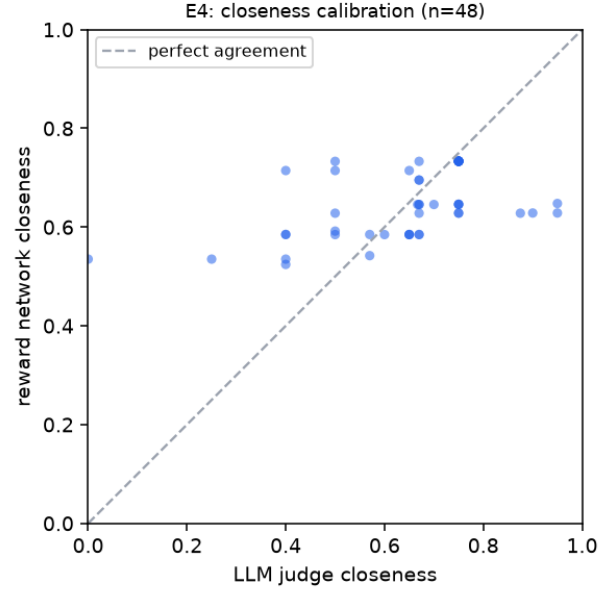


Fig. 3. E4: closeness rubric score, real LLM judge vs. distilled reward network, on all 48 held-out situations.

whichever reference it is steered toward, including one that is demonstrably suboptimal. This is a direct empirical illustration of why iterative-refinement alignment is only as trustworthy as the reference signal used to drive it.

D. E4: distilling the LLM judge into a reward network

The distilled reward network reaches in-sample rubric mean absolute error (MAE) 0.069 on its 258-example training pool and, on 48 held-out situations each checked against a fresh real LLM call, tracks the LLM’s rubric scores nearly as closely out of sample (MAE 0.058, CI [0.048, 0.068]), indicating it has not simply memorised the training situations. At the same time it runs at a median 2.4 ms latency versus 2.44 s for a real LLM call - roughly an 1800 $\times$ speedup (we use the median because a handful of calls on both sides of this comparison hit transient multi-second network delays that would otherwise let a single outlier dominate a mean). However, the network’s argmax action matches the LLM judge’s argmax action only 48% of the time (CI [34%, 62%], JS divergence 0.232 on the same held-out set), markedly worse than the LLM-vs-rule agreement measured directly elsewhere in this study (JS divergence 0.086-0.095, Table I). This is, however, a substantial improvement over a smaller 41-pair pilot run of the same pipeline, which reached only 25% argmax agreement - suggesting the gap narrows, without yet closing, as more distillation data becomes available. The closeness rubric tracks the real judge well in aggregate (Fig. 3), yet the softmax used to turn predicted closeness into an action distribution is sensitive to exactly the small regression errors that a rubric-level MAE of 0.058 permits, because neighbouring cells’ true closeness values are often close together. A deployment decision based only on rubric MAE would miss this gap.

TABLE I

CROSS-EXPERIMENT SUMMARY ($n = 64$ FOR E1-E3, $n = 48$ HELD-OUT FOR E4; BRACKETS ARE 95% CIs). "ARGMAX AGREE." IS THE FRACTION OF SITUATIONS WHERE THE TWO COMPARED POLICIES PICK THE SAME BEST ACTION; JS DIV. IS THE JENSEN-SHANNON DIVERGENCE BETWEEN THEIR ACTION DISTRIBUTIONS; RUBRIC MAE IS AVERAGED OVER THE CLOSENESS, AVOIDED-DAMAGE, AND PATH-EFFICIENCY FIELDS.

Exp.	Reference	Instr.	Iters.	JS div.	Argmax
E1	Rule-based	No	1	.095 [.06,.13]	.78 [.67,.87]
E2	Rule-based	Yes	1.34/4	.017 [.01,.02]	1.00 [.94,1]
E3	NN (72%)	Yes	1.69/4	.011 [.008,.014]	.86 [.75,.92]
E4	Distilled net	Yes (tr.)	-	.232 [.20,.26]	.48 [.34,.62]

E. Analysis of results

Three patterns emerge from Table I, all backed by 95% confidence intervals rather than single point estimates from a handful of trials. First, agreement between a real LLM judge and a geometric reference is already high without instruction or feedback (E1: 78% argmax agreement, CI [67%, 87%]), and its error mode is localised and correctable rather than diffuse. Second, one or two rounds of feedback closes most of the remaining gap (E2), but the same mechanism just as readily aligns the judge with an imperfect reference if that is what it is steered toward (E3), at a measurably higher iteration count (1.69 vs. 1.34, non-overlapping CIs): convergence rate and final divergence measure *steerability*, not correctness, and should be interpreted as such. Third, distillation (E4) exposes a gap between two fidelity measures often reported as interchangeable: rubric-score agreement (MAE 0.058) and action-level agreement (48% argmax match) can diverge sharply for the same model on the same data, because a small regression error is amplified by the softmax that converts predicted scores into a decision - a gap a rubric-error-only evaluation would not surface, and one that a $6\times$ increase in distillation data (from a 41-pair pilot to this 258-pair run) narrowed but did not close.

VII. CONCLUSION

We compared three sources of reward for a language-conditioned robot navigation task - a hand-written rule, a real LLM-as-judge reward, and a network distilled from that judge's outputs - on one shared environment, through four controlled experiments. The results support three practical conclusions. A real LLM judge, even zero-shot and without instructions, already agrees with a hand-written geometric policy on the correct action most of the time, and its errors are concentrated and diagnosable rather than pervasive (E1). Giving the judge language and a concrete, iterative feedback signal is a cheap way to close most of the remaining gap, but this alignment tracks whichever reference policy is used to generate the feedback, correct or not (E2, E3) - so the choice of reference used for iterative refinement deserves as much scrutiny as the judge itself. And replacing the judge with a distilled reward network trades a large latency cost for a real, but partial, fidelity cost: rubric-level and action-level agreement should be measured and reported separately,

because a model that looks faithful by one measure can look substantially less faithful by the other (E4).

This study trades training-pipeline scale - no PPO, no sentence-transformer instruction encoder, no full MiniGrid turn-based action space - for the ability to run every reward source for real, including genuine LLM calls, within one tightly controlled comparison. Every experiment now evaluates 64 situations (48 held-out for E4), drawn from every eligible cell of the map with several independent rollout replicates each, which is enough for the 95% confidence intervals reported throughout to make results such as E2 vs. E3's iteration counts statistically distinguishable rather than an artifact of a small sample. The main remaining limitation is that all four experiments share a *single* map, so the numbers characterise this map's topology - its particular wall-gap placements, especially - without yet showing they generalise across layouts. Natural next steps include repeating E1-E3 across multiple, independently generated maps to test whether the findings are map-specific or general; training the distilled network on substantially more data to test how far the E4 argmax-agreement gap can be closed, building on the 25%→48% improvement already seen between a 41-pair pilot and this 258-pair run; replacing the automated references with human preference labels to test whether iterative refinement also aligns a judge to people; and folding the full PPO/sentence-transformer/MiniGrid pipeline back in once a reward source has been selected using this lighter-weight comparison.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [2] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in ai safety," *arXiv preprint arXiv:1606.06565*, 2016.
- [3] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," in *International Conference on Learning Representations (ICLR)*, 2024.
- [4] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu, "Text2reward: Automated dense reward function generation for reinforcement learning," in *International Conference on Learning Representations (ICLR)*, 2024.
- [5] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [6] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging llm-as-a-judge with mt-bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [7] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, 1999.
- [8] M. Chevalier-Boisvert, D. Bahdanau, S. Lahlou, L. Willems, C. Saharia, T. H. Nguyen, and Y. Bengio, "BabyAI: A platform to study the sample efficiency of grounded language learning," in *International Conference on Learning Representations (ICLR)*, 2019.
- [9] M. Chevalier-Boisvert, B. Dai, M. Towers, R. de Lazcano, L. Willems, S. Lahlou, S. Pal, P. S. Castro, and J. Terry, "Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.
- [10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

- [11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [12] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dornmann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.